



دانشگاه شهیدچمران اهواز

Shahid Chamran

University of Ahvaz

دانشکده مهندسی

گروه مهندسی برق

گزارش پروژه درسی سیستم دیجیتال ۲

پروژه با میکروکنترلر AVR خانواده ATmega و کدنویسی با آتمل استودیو و شبیه سازی در پروتئوس

عنوان پروژه:

مسافت سنج آلتراسونیک و کنترل و مانیتورینگ دو طرفه با بلوتوث

دانشجویان:

محمد رضا مدنیان محمدی (۴۰۰۶۵۵۱۰۹) ، محمد طاهای غلامی (۴۰۰۶۵۵۰۶۸) ، بهراد

صدری پور (۴۰۰۶۵۵۰۵۸)

نام استاد درس:

دکتر نوید علایی شینی

تاریخ تحویل گزارش:

خرداد ماه ۱۴۰۳

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

چکیده

در این پروژه میکروکنترلر Atmega16a به عنوان هسته اصلی پروژه انتخاب شده است و از امکانات مورد استفاده این میکروکنترلر در این پروژه میتوان به این موارد اشاره کرد: استفاده از Timer0 ، وقفه (Interrupt) و ارتباط سریال (UART). از استفاده همزمان تایمر ۰ و وقفه برای تنظیم کردن زمان بوق زدن بازار و زمان فعال شدن رله استفاده شده و همچنین از وقفه برای برای بروزرسانی LCD و ارسال و دریافت داده بوسیله بلوتوث استفاده شده است. در کل این پروژه در دو مد کاری کار می کند : ۱- مد التراسونیک ۲- مد بلوتوث در مد التراسونیک رله با استفاده از فاصله فعال و یا غیر فعال می شود ، و در مد بلوتوث رله با استفاده از بلوتوث کنترل می شود. سختی که در این پروژه وجود داشت در سوییچ شدن بین این دو مد بود. در واقع هدف این بود که بتوانیم با دستورات ارسالی از طریق بلوتوث بین دو مد التراسونیک و بلوتوث جا به جا شویم. و راه حلی که توانستیم برای این مشکل پیدا کنیم استفاده از قابلیت STAT MACHINE در زبان برنامه نویسی C بود که در کد با Enum نوشته شده است.

کاربرد کلی این پروژه این است که به عنوان یک ابزار حفاظتی- ایمنی در صنعت استفاده می شود، به عنوان مثال کاربرد صنعتی آن این است که در ماشین ها استفاده می شود و اینگونه عمل می کند که اگر ماشین x نزدیک جسمی شود و فاصله آن کمتر از حد مشخصی شود (نزدیک برخورد با آن) ماشین x به صورت خودکار ترمز کرده و سرعت خود را کم می کند و همچنین در صفحه نمایش ماشین به راننده هشدار می دهد.

فهرست مطالب

۱	فصل اول نرم افزار
۲	توضیح خط به خط کد پروژه
۲۰	فصل دوم سخت افزار
۲۱	شماتیک مدار در پروتئوس
۲۳	عکس از مدار بسته شده روی برد مورد
۲۴	فصل سوم نتایج
۲۵	عکس نتایج بدست آمده از مدار پروژه
۲۹	پیوست

نکته: معرفی قطعات استفاده شده در پروژه در فایل جداگانه به صورت پاورپوینت قرار داده شده است

فصل اول

نرم افزار

```

1)#include <avr/io.h>
2)#include <util/delay.h>
3)#include "LCD.h"
4)#include <stdlib.h> //dtostrf
5)#include <stdio.h> //sprintf function
6)#include <avr/interrupt.h>

```

از خط ۱ تا ۶ کتابخانه ها را تعریف کردیم . در خط ۳ کتابخانه مربوط به lcd کارکتری ۲ در ۱۶ را اضافه کردیم . در خط ۴ کتابخانه ای که اضافه کردیم برای استفاده از تابع dtostrf است که این تابع متغیر از نوع float (اعداد اعشاری) را به رشته تبدیل می کند تا بتوانیم آن را در lcd نمایش بدهیم. در خط ۵ کتابخانه ای که اضافه کردیم برای استفاده از تابع sprintf است که این تابع اعداد از نوع int (اعداد صحیح) را به رشته تبدیل می کند دلیل استفاده از این تابع را در خط ۸۸ توضیح دادم

```

7)#define F_CPU 8000000UL
8)#define BUZZER_PIN PORTD7
9)#define RELAY_PIN PORTC0
10)#define TRIG_PIN PORTA0
11)#define ECHO_PIN PORTD6
12)#define USART_BAUDRATE 9600
13)#define BAUD ((F_CPU / (USART_BAUDRATE*16UL))-1)

```

از خط ۷ تا ۱۳ ما ثابت هایی را که در طول برنامه می خواهیم از آن ها استفاده کنیم و همچنین در طول برنامه تغییر نمی کنند را اینجا نوشتیم. در خط ۷ ما کلاک cpu را ۸ مگاهرتز گذاشتیم. در خط ۸ و ۹ پورت های D7 و C0 میکرو را به ترتیب به بازر و رله اختصاص دادیم. در خط ۱۰ و ۱۱ پورت ها A0 و D6 میکرو را به ترتیب به پایه های تریگر و اکو ماژول التراسونیک اختصاص دادیم .

در خط ۱۲ بادریت ارتباط سریال را به ۹۶۰۰ تنظیم کردیم (دلیل اینکه به ۹۶۰۰ تنظیم کردیم این است که ماژول بلوتوث HC-05 از ارتباط UART و با بادریت ۹۶۰۰ پشتیبانی می کند.) در خط ۱۳ با استفاده از بادریت UBRR را محاسبه میکنیم و آن را در متغیر BAUD ذخیره می کنیم که مقدار آن برابر ۵۱ می شود.

```

14)enum {menu1,menu2}menu;
15)float distance;
16)int duration;

```

```

17)char string[20];
18)volatile int received = 0;
19)volatile int overflow_counter = 0;
20)volatile int check = 0;
21)volatile char received_char = '0';
22)volatile char transmit_buffer[256];
23)volatile uint8_t transmit_read_pos = 0;
24)volatile uint8_t transmit_write_pos = 0;
25)volatile int prev_received = 0;

```

در خط های ۱۴ تا ۲۵ متغیر ها را تعریف کردیم. در خط ۱۴ از enum استفاده کردیم. در برنامه نویسی C، نوع enum یا شمارشی نوعی داده تشکیل شده از ثابت های عددی است. در واقع متغیر های داخل enum (menu1 , menu2) مقدار های ثابتی به آن ها اختصاص داده می شود و مقدار آن ها تغییر نمی کند مگر اینکه ما به آن مقدار های مشخصی بدهیم، در این برنامه با ترکیب enum و switch,case میخواستیم با سرعت بیشتری به حالت های مختلف توسط مازول بلوتوث برویم (همچنین enum ها گزینه ی بسیار خوبی برای کار با flag ها هستند).

در خط ۱۵ متغیر distance را برای قرار دادن مقدار فاصله در آن تعریف کردیم.

در خط ۱۶ متغیر duration را برای قرار دادن مقدار زمان رفت و برگشت فاصله در آن تعریف کردیم.

در خط ۱۷ متغیر string را برای نمایش مقدار فاصله در lcd تعریف کردیم.

در خط ۱۸ متغیر received برای دریافت کاراکتر های ارسالی از سمت فرستنده است در واقع کاراکتر هایی را که فرستنده ارسال می کند را به صورت عدد صحیح در خود ذخیره می کند (پایین تر خواهیم گفت که چگونه به صورت عدد صحیح در آن ذخیره می شود).

تعریف متغیر از نوع Volatile: متغیرهایی از نوع Volatile متغیرهایی هستند که ممکن است مقدار

آنها توسط یک پردازش خارجی تغییر یابد. این پردازش می تواند وقوع یک وقفه، یا تغییر از طریق یک پردازش موازی باشد. کامپایلرهای زبان برنامه نویسی C اغلب از روش های بهینه سازی برای دستیابی به متغیرهای تعریف شده در طول برنامه استفاده می کنند. Volatile باعث می شه که از رجیستر های CPU استفاده

نکنه و متغیر رو در فضای SRAM قرار بده قرار دادن Volatile در هنگام تعریف متغیر باعث میشه با هر بار استفاده از متغیر ، کامپایلر مستقیماً از فضای اختصاص داده شده از SRAM مقدار جدید رو بگیره و تغییرات مقدار متغیر در قسمتهای دیگه برنامه تشخیص داده بشه. Volatile رو زمانی بهتره استفاده کنیم که بخواهیم یک متغیر عمومی رو توی برنامه سرویس وقفه مقدار دهی کنیم .

در خط ۱۹ متغیر overflow_counter را برای وقفه تایمر * استفاده می کنیم.

در خط ۲۰ متغیر check برای به روز رسانی lcd استفاده می شود.

در خط ۲۱ متغیر received_char برای دریافت کاراکتر های ارسالی از سمت فرستنده است و به صورت کاراکتر هم ذخیره می شود.

در خط ۲۲ متغیر transmit_buffer[256] در واقع کاراکتر های ارسالی و دریافتی را در خود ذخیره می کند

در خط ۲۳ متغیر transmit_read_pos به عنوان اندیس خواندن در transmit_buffe عمل می کند.

وقتی یک کاراکتر از بافر برای ارسال از طریق پورت سریال آماده می شود، transmit_read_pos نشان دهنده موقعیت کاراکتر بعدی در بافر است که باید ارسال شود.

در خط ۲۴ متغیر transmit_write_pos به عنوان اندیس نوشتن در بافر transmit_buffe عمل می کند. هر زمان که یک کاراکتر جدید برای ارسال به بافر اضافه می شود، transmit_write_pos نشان دهنده موقعیتی است که کاراکتر جدید باید در آن قرار گیرد.

در خط ۲۵ متغیر prev_received برای این است که کرای کنیم که در حلقه رشته ای که از طریق بلوتوث دریافت می کنیم به صورت بینهایت برای من ارسال نشود و فقط یک بار ارسال شود (پایین تر بیشتر توضیح خواهیم داد) .


```

26) void init_ports() {
27)   DDRA |= (1 << TRIG_PIN); // Set TRIG_PIN as output on PORTA
28)   DDRD |= (1 << BUZZER_PIN); // Set BUZZER_PIN as output on PORTD
29)   DDRD &= ~(1 << ECHO_PIN); // Set ECHO_PIN as input on PORTD
30)   PORTD |= (1 << ECHO_PIN); // Pull-up on ECHO_PIN
31)   DDRC |= (1<<RELAY_PIN); // Set RELAY_PIN as output on PORTC
}

```

در خط ۲۶ تابع init_port را تعریف کردیم که در این تابع پیکربندی های پورت ها رو انجام دادیم .
توضیح هر پورت در روبه روی آن نوشته شده است) .

```

32) void trigger_sensor() {
33)   PORTA |= (1 << TRIG_PIN);
34)   _delay_us(10);
35)   PORTA &= ~(1 << TRIG_PIN);
}

```

در خط ۳۲ تابعی تعریف کردیم که در این تابع پایه ی تریگر سنسور التراسونیک را پیکر بندی کردیم. در سنسور التراسونیک پایه تریگر به صورت خروجی تنظیم شده است . پین تریگر فعال کننده بخش فرستنده ماژول التراسونیک است که با تحریک این پایه به مدت ۱۰ میکروثانیه پالسی با قدرت ۴۰ کیلوهرتز برای اندازه گیری ارسال می شود.

```

36) unsigned int read_sensor() {
37)   unsigned int count = 0;
38)   while (!(PIND & (1 << ECHO_PIN))); // Wait for the rising edge
39)   while (PIND & (1 << ECHO_PIN)) // Measure the width of the high pulse
40)   {
41)       count++;
42)       _delay_us(0.3);
43)   }
44)   return count;
45) }

```

در خط ۳۶ تابعی که تعریف کردیم برای اندازه گیری زمان رفت و برگشت پالسی است که توسط پایه تریگر ارسال شده است. در خط ۳۷ متغیری که تعریف کردیم مقدار زمان را در خودش ذخیره می کند. در خط ۳۸ حلقه نوشتیم که معنی آن این است که تا زمانی که پین اکو صفر است در این حلقه بمان و هیچ کاری نکن (در واقع وقتی پین تریگر پالسی را ارسال می کند پین اکو همان لحظه یک می شود و تا زمانی یک می ماند که پالس ارسالی ، دریافت شود. زمانی که آن پالس دریافت می شود پین اکو صفر می شود و متوجه می شویم که پالس دریافت شده است)

در خط ۳۹ تا ۴۱ معنی حلقه این است که تا زمانی که پین اکو یک است هر ۰.۳ میکروثانیه یک واحد به متغیر count اضافه شود. (در واقع درستش اینه که هر یک میکروثانیه به متغیر اضافه شود ولی به دلیل تاخیر هایی که داخل برنامه وجود دارد به صورت عملی و با چند بار آزمایش فهمیدیم که هر ۰.۳ میکروثانیه فاصله را به درستی اندازه گیری می کند.) (دلیل علمی پایین تر گفته خواهد شد)

```
43) void init_timer0() {
44)   TCCR0 |= (1 << CS02) | (1 << CS00);
45)   TIMSK |= (1 << TOIE0);
46)   sei();
}
```

در خط ۴۳ پیکربندی تایمر ۰ را انجام دادیم. در خط ۴۴ با توجه به بیت هایی که یک شده مقسم فرکانسی تایمر ۰ را ۱۰۲۴ گرفتیم.

در خط ۴۵ این بیت را یک کردیم که با سرریز تایمر ۰ (که ۲۵۵ است) وقفه سرریز تایمر ۰ فعال شود. در خط ۴۶ با این کد وقفه های عمومی را فعال کردیم.

```
47) ISR(TIMER0_OVF_vect) {
48)   overflow_counter++;
49)   check++;
}
```

تابع فعال شدن وقفه با سرریز تایمر ۰ در خط ۴۷ آورده شده است (یعنی هر بار که تایمر صفر سرریز یا همان به ۲۵۵ می رسد این تابع فراخوانی می شود و دستورات داخل این تابع اجرا می شوند).

در خط ۴۸ با هر بار سرریز شدن تایمر ۰ یک واحد به متغیر overflow_counter اضافه شود. (با ای متغیر می خواهیم صدای بوق بازر و همچنین زمان فعال شدن رله را کنترل کنیم)

در خط ۴۹ با هر بار سرریز شدن تایمر ۰ یک واحد به متغیر check اضافه می شود. (پایین تر خواهیم گفت که چگونه از این متغیر برای به روز رسانی lcd استفاده می شود).

```

50) ISR(USART_RXC_vect) {
51) received_char = UDR;
52) switch (received_char){
53)     case '1':
54)         received = 1;
55)         prev_received = 1;
56)         break;
57)     case '2' :
58)         received = 2;
59)         prev_received = 1;
60)         break;
61)     case '3' :
62)         received = 3;
63)         prev_received = 1;
64)         break;
65)     case '0' :
66)         received = 0;
67)         break;
        }
    }
}

```

تابع وقفه دریافت کاراکتر در ارتباط سریال در خط ۵۰ مشاهده می شود. (زمانی این تابع وقفه اجرا می شود که عملیات دریافت کامل شود، وقتی عملیات دریافت کامل شود در رجیستر UCSRA بیت RXC می شود و با یک کردن بیت RXCIE در رجیستر UCSRB باعث اجرای تابع وقفه می شود).
در خط ۵۱ دیتایی که در رجیستر UDR ذخیره شده است در متغیر received_char می ریزیم.
در خط ۵۲ یک تابع از نوع switch می نویسیم که بر اساس کاراکترهای دریافتی عملیات های مختلفی انجام دهد.

در خط ۵۳ تا ۶۴ اگر ما کاراکتر ۱ را ارسال کنیم در متغیر received و prev_received عدد ۱ را قرار می دهیم. و دستور break به معنای خارج شدن از case مورد نظر است ، همچنین مشاهده می کنیم با ارسال کاراکترهای بعدی در متغیر received همان عدد ذخیره می شود اما از نوع int . (در جلوتر خواهیم گفت که با ارسال این اعداد چه کاری انجام خواهد شد)
در خط ۶۵ با ارسال کاراکتر ۰ متغیر received مقدار ۰ را خواهد گرفت و با ارسال عدد ۰ ما به مد التراسونیک خواهیم رفت (در این پروژه ما دو مد کاری داریم : ۱ - مد التراسونیک : که رله با استفاده از فاصله کنترل می شود و ۲ - مد بلوتوث : که رله با بلوتوث کنترل می شود.)

```

68) ISR(USART_UDRE_vect) {
69) if (transmit_read_pos != transmit_write_pos) {
70)    UDR = transmit_buffer[transmit_read_pos];
71)    transmit_read_pos++;

72)    if (transmit_read_pos >= sizeof(transmit_buffer)) {
73)        transmit_read_pos = 0;    }    }
74) else {
75)    UCSRB &= ~(1 << UDRIE);
    }
}

```

تابع وقفه UDRE در خط ۶۸ مشاهده می شود. زمانی این تابع وقفه فراخوانی می شود که رجیستر UDR خالی باشد (در واقع با خالی شدن رجیستر UDR بیت UDRE واقع در رجیستر UCSRA یک می شود و با فعال کردن بیت UDRIE واقع در رجیستر UCSRB باعث فراخوانی این تابع می شود)

تابع if در خط ۶۹ به این معنی است که اگر موقعیت خواندن با موقعیت نوشتن برابر نباشد ، یعنی داده ای در بافر وجود دارد که هنوز ارسال نشده است. پس وقتی داده ای برای ارسال وجود داشته باشد این تابع if اجرا می شود.

در خط ۷۰ داده ای که در بافر وجود دارد را در رجیستر UDR ذخیره می کنیم. و در خط ۷۱ پس از ارسال هر کاراکتر، این اندیس یک واحد افزایش می یابد تا به کاراکتر بعدی در بافر اشاره کند. (اگر اندیس افزایش نیابد، ممکن است داده های جدید بر روی داده های قدیمی تر که هنوز ارسال نشده اند، نوشته شوند و در نتیجه داده ها از دست بروند.)

در خط ۷۲ تابع if به این معناست که اگر موقعیت خواندن به انتهای بافر رسید . دستورات شرط اجرا شود که یک دستور بیشتر ندارد و آن هم در خط ۷۳ نوشته شده و دستور این است که موقعیت خوندن به ابتدای بافر برگردد.

در خط ۷۴ دستور else به این معنی است که اگر شرط خط ۶۹ برقرار نبود (یعنی موقعیت خواندن و نوشتن با هم برابر بود) بیايد دستور خط ۷۵ را اجرا کند که دستور خط ۷۵ به این معنی است که بیت UDRIE واقع در رجیستر UCSRB را صفر کن. (اگر transmit_read_pos و transmit_write_pos برابر باشند، به این معنی است که تمام داده های بافر ارسال شده اند و دیگر داده ای برای ارسال وجود ندارد. در این حالت، وقفه UDRE غیرفعال می شود تا از اجرای بی مورد ISR جلوگیری شود).

```
76) void UART_Txchar(char ch) {  
77)   transmit_buffer[transmit_write_pos] = ch;  
78)   transmit_write_pos++;
```

```

79) if (transmit_write_pos >= sizeof(transmit_buffer)) {
80)     transmit_write_pos = 0;
    }

```

```

81) UCSRB |= (1 << UDRIE);
    }

```

تابع ارسال کاراکتر در خط ۷۶ قابل مشاهده است. در خط ۷۷ کاراکتر را در موقعیت نوشتن قرار می دهیم (منظور کاراکتری است که می‌خواهیم برای ما ارسال شود) و در خط ۷۸ موقعیت (اندیس) نوشتن را یک واحد افزایش می دهیم تا برای دریافت کاراکتر بعدی آماده باشد. در خط ۷۹ تابع if به این معنی است که اگر موقعیت (اندیس نوشتن) به پایان بافر رسید دستور خط ۸۰ را اجرا کند که یعنی موقعیت (اندیس) خواندن را به ابتدای بافر برگرداند. در خط ۸۱ ما بیت UDRIE را فعال (۱) می کنیم تا وقفه خالی بودن رجیستر UDR فعال شود. (در واقع وقتی رجیستر UDR خالی می شود فلگ (پرچم) UDRE واقع در رجیستر UCSRA فعال میشود. با فعال شدن این بیت و یک کردن بیت UDRIE میتوانیم کاراکتر بعدی را ارسال کنیم).

```

82) void UART_Sendstring(char *str) {
83)     while (*str) {
84)         UART_Txchar(*str++);
    }
}

```

تابع ارسال رشته در سطر ۸۲ قابل مشاهده است. ما قصد داریم با استفاده از این تابع رشته هایی از بلوتوث برای ما ارسال شود. در سطر ۸۳ حلقه while به این معنی است که تا زمانی که به کاراکتر پایان رشته (null terminator) نرسیده‌ایم، دستور خط ۸۴ اجرا شود، در دستور خط ۸۴ تابع UART_Txchar فراخوانی می‌شود که یک کاراکتر را به عنوان ورودی می‌گیرد و آن را ارسال می‌کند. این تابع با استفاده از عملگر * کاراکتر فعلی را که str به آن اشاره دارد، دریافت می‌کند. سپس با استفاده از str++، اشاره گر به کاراکتر بعدی در رشته افزایش می‌یابد. (در واقع این حلقه تا زمانی که به آخر رشته

نرسیده است دستور دریافت کاراکتر را اجرا می کند و دونه به دونه از کاراکتر های دریافتی را ارسال می کند تا به صورت یک رشته ارسال شود).

```
85) void UART_SendVariableValue(unsigned int variable) {  
86)   char buffer[10];  
87)   sprintf(buffer, "%d", variable); // Convert variable value to string  
    and store in buffer  
88)   UART_Sendstring(buffer); // Send string by UART_SendVariableValue  
    Function  
}
```

تابع ارسال متغیر در سطر ۸۵ قابل مشاهده است . هدف ما از ایجاد این تابع این است که علاوه بر اینکه بتوانیم برای فرستنده رشته ارسال بکنیم بتوانیم متغیر هم ارسال بکنیم. در خط ۸۶ متغیری از نوع کاراکتر به نام بافر ایجاد کردیم که بتوانیم متغیر مورد نظر را در آن قرار دهیم.

تابع sprintf برای این است که متغیر (از نوع int) به رشته تبدیل شود. دلیل تبدیل ما این است که در پروتکل ارتباطی سریال نمی توان داده هایی از نوع float و int و ... را ارسال کرد و حتما باید آن ها را به کارکتر و یا رشته تبدیل بکنیم.

در خط ۸۸ متغیر ما که در بافر قرار گرفته با استفاده از تابع UART_Sendstring که ان را در مرحله قبل پیکربندی کردیم ارسال می شود

```
89) void UART_Init() {  
90)   UCSRB |= (1 << RXEN) | (1 << TXEN);  
91)   UCSRC |= (1 << URSEL) | (1 << UCSZ0) | (1 << UCSZ1);  
92)   UBRRL = BAUD;  
93)   UBRRH = (BAUD >> 8);  
  
94)   UCSRB |= (1 << RXCIE);  
95)   UCSRB |= (1 << UDRIE);
```

}

با تابع خط ۸۹ رجیستر های ارتباط UART پیکربندی شده اند.

با یک کردن بیت های خط ۹۰ دریافت و ارسال داده را فعال کردیم.

با یک کردن بیت URSEL در خط ۹۱ از بین دو رجیستر UCSRC و UBBRH رجیستر UCSRC را انتخاب می

کنیم. و با یک کردن بیت های UCSZ0 و UCSZ1 سائز بیت های ارسال و دریافت را ۸ بیت در نظر می

گیریم.

در خط های ۹۲ و ۹۳ مقدار BAUD به دست آمده (در خط ۱۲ بدست آوردیم) در رجیستر UBRRL قرار

می دهیم و همچنین بقیه ی بیت های آن را در رجیستر UBRRH قرار می دهیم (چون رجیستر UBRR یک

رجیستر 12 بیتی است به دو رجیستر UBRRH و UBRRL تقسیم شده است) (البته چون مقدار UBRR برابر

۵۱ شده است و سائز آن کمتر از ۸ بیت است می توان آن را فقط در رجیستر UBRRL انتقال داد و در

رجیستر UBRRH مقدار صفر قرار داد . ولی چون به صورت کلی اینگونه نوشته می شود من اینگونه نوشتم).

```
96) int main(void) {
97) init_ports();
98) init_timer0();
99) UART_Init();
100) LCD_Init();
101) PORTD |= (1<<BUZZER_PIN);
```

در تابع اصلی برنامه ما در خط های ۹۷ تا ۱۰۰ به ترتیب تابع های پیکربندی پورت ها ، پیکربندی رجیستر

های تایمر 0 ، پیکربندی رجیستر های ارتباط UART و تابع آماده سازی LCD را فراخوانی کردیم تا در برنامه

اصلی اعمال شوند. همچنین در خط ۱۰۱ ما ابتدا به طور پیش فرض پورت بازر را ۱ (۵ ولت) قرار دادیم.

```
102) while (1) {
103) trigger_sensor();
104) duration = read_sensor();
```



```
105) distance = (duration * (0.0343)) / 2;
```

در خط ۱۰۳ ما تابع پیکربندی پین تریگر التراسونیک را در حلقه while اصلی برنامه قرار دادیم که با هر بار اجرای حلقه پین تریگر التراسونیک در زمان حدود ۱۰ میکروثانیه هشت پالس برای اندازه گیری فاصله بفرستد.

در خط ۱۰۴ مقدار زمانی که از تابع read_sensor() بدست می آید را در داخل متغیری به نام duration انتقال می دهیم.

در خط ۱۰۵ فاصله را با زمانی که در متغیر duration (بر حسب میکروثانیه) ذخیره شده است بدست می آید (در واقع فاصله از طریق سرعت صوت بدست می آید. که فرمول بدست آوردن فاصله به صورت زیر است:

سرعت صوت (0.0343 سانتی متر بر میکروثانیه) * زمان (بر حسب میکروثانیه) = ۲ × فاصله (بر حسب سانتی متر)

```
106) switch(menu){
```

```
107) case menu1:
```

```
108)   if (distance > 70) {
```

```
109)     PORTD &= ~(1 << BUZZER_PIN);
```

```
110)     PORTC &= ~(1<<RELAY_PIN);
```

```
111)     overflow_counter = 0;
```

```
112)   } else if (distance <= 70 && distance > 55) {
```

```
113)     if (overflow_counter >= 30) { // 1 second
```

```
114)       PORTD ^= (1 << BUZZER_PIN);
```

```
115)       PORTC &= ~(1<<RELAY_PIN);
```

```
116)       overflow_counter = 0;
```

```
       }
```

```
117)   } else if (distance <= 55 && distance > 30) {
```

```
118)     if (overflow_counter >= 15) { // 0.5 second
```

```
119)       PORTD ^= (1 << BUZZER_PIN);
```

```
120)       PORTC &= ~(1<<RELAY_PIN);
```

```

121) overflow_counter = 0;
        }
122) } else if (distance <= 30 && distance > 10) {
123) if (overflow_counter >= 3) { // 0.1 second
124) PORTD ^= (1 << BUZZER_PIN);
125) PORTC &= ~(1<<RELAY_PIN);
126) overflow_counter = 0;
        }
127) } else if (distance <= 10) {
128) PORTD |= (1 << BUZZER_PIN);
129) if(overflow_counter >= 61){
130) PORTC |= (1<<RELAY_PIN);
131) overflow_counter = 0;
        }
        }
132) if(received != 0){
133) menu=menu2;
        }
134) break;

```

در خط ۱۰۶ تابع switch را برای انتقال بین دو مد التراسونیک (menu1) و بلوتوث (menu2) تعریف می کنیم.

در خط ۱۰۷ دستورات case menu1 همان دستورات مد التراسونیک است.

در خط ۱۰۸ تا ۱۱۱ شرطی که داریم به این معناست که اگر فاصله تا جسمی بیش از ۷۰ سانت بود بازر و رله فعال نشوند (به ترتیب خط ۱۰۹ و ۱۱۰) و همچنین در خط ۱۱۱ گفتیم مقدار overflow_counter ریست شود تا برای محاسبه زمان در شرط های بعدی آماده شود.

در خط ۱۱۲ تا ۱۱۶ گفتیم اگر فاصله بین ۵۵ تا ۷۰ سانتی متر بود بیا بعد از ۱ ثانیه بازر را خاموش و روشن کن و همچنین رله همچنان خاموش بماند و متغیر overflow_counter صفر شود که شمارش های

بعدی از نو آغاز شود. (در واقع با فرمول زیر میتوان زمان دلخواه را با تایمر صفر محاسبه کرد:

$$\text{overflow_counter} = \text{Time} * ((F_CPU)/(256 * 1024))$$

در فرمول F_CPU فرکانس میکرو هست که برابر ۸ مگاهرتز است. و همچنین ۲۵۶ تعداد شمارش تایمر ۰

هست که از ۰ تا ۲۵۵ میشمارد و در ۲۵۵ باعث سرریز شدن آن می شود. و عدد ۱۰۲۴ مقسم فرکانسی

تایمر ۰ است که ما آن را ۱۰۲۴ در نظر گرفتیم. و متغیر Time همان زمان مورد نظر است که هر چند ثانیه

بازر بوق بزند. اگر در فرمول به جای متغیر Time یک ثانیه قرار دهیم مقدار overflow_counter تقریباً برابر

۳۱ می شود، همانطور که در خط ۱۱۳ هم مشاهده می شود نوشتن هر موقع overflow_counter بیشتر یا

مساوی ۳۱ شد بازز نقیض شود (یعنی اگر روشن بود خاموش شود و برعکس) (در واقع ^= نماد XOR

است).

در خط های ۱۱۷ تا ۱۲۱ شرط این است که اگر فاصله بین ۳۰ تا ۵۵ باشد هر ۰.۵ ثانیه بازز بوق بزند و

همچنان رله خاموش بماند و در خط ۱۲۲ مقدار overflow_counter را صفر کردیم که برای شمارش های

بعد از نو آغاز شود. (اگر در فرمول بالا به جای Time مقدار ۰.۵ ثانیه را قرار دهیم مقدار

overflow_counter تقریباً برابر ۱۵ خواهد شد).

در خط های ۱۲۲ تا ۱۲۶ شرط این است که اگر فاصله بین ۱۰ تا ۳۰ سانتی متر باشد هر ۰.۱ ثانیه بازز

بوق بزند و همچنان رله خاموش بماند. و مقدار overflow_counter ریست شود. (اگر در فرمول بالا به جای

Time مقدار ۰.۱ ثانیه را قرار دهیم مقدار overflow_counter تقریباً برابر ۳ خواهد شد).

در خط های ۱۲۷ تا ۱۳۱ شرط این است که اگر فاصله کمتر از ۱۰ سانتی متر بود بازز به صورت ممتد بوق

بزند و رله بعد از ۲ ثانیه فعال شود و دوباره مقدار overflow_counter ریست شود. ((اگر در فرمول بالا به

جای Time مقدار ۲ ثانیه را قرار دهیم مقدار overflow_counter تقریباً برابر ۶۱ خواهد شد).

در خط های ۱۳۲ و ۱۳۳ شرط این است که اگر عددی غیر 0 را ارسال کردیم به مد بلوتوث برود (

menu2) (در واقع ما از طریق ترمینال بلوتوث کاراکتر ها را ارسال می کنیم ولی چون در تابع ISR خط

۵۱ کاراکترهای ۰ و ۱ و ۲ و ۳ را به عدد تبدیل کردیم ، دیگر آن را به صورت عددی بیان می کنیم). در خط ۱۳۴ دستور break باعث خارج شدن از مد التراسونیک می شود).

```
135) case menu2:
136) if (distance > 70) {
137) PORTD &= ~(1 << BUZZER_PIN);
138) overflow_counter = 0;
139) } else if (distance <= 70 && distance > 55) {
140) if (overflow_counter >= 31) { // 1 second
141) PORTD ^= (1 << BUZZER_PIN);
142) overflow_counter = 0;
        }
143) } else if (distance <= 55 && distance > 30) {
144) if (overflow_counter >= 15) { // 0.5 second
145) PORTD ^= (1 << BUZZER_PIN);
146) overflow_counter = 0;
        }
147) } else if (distance <= 30 && distance > 10) {
148) if (overflow_counter >= 3) { // 0.1 second
149) PORTD ^= (1 << BUZZER_PIN);
150) overflow_counter = 0;
        }
151) } else if (distance <= 10) {
152) PORTD |= (1 << BUZZER_PIN);
        }
153) if(prev_received == 1){
154) if (received == 1){
155) PORTC |= (1<<RELAY_PIN);
156) UART_Sendstring("Relay ON");
        }
157) else if (received == 2){
```

```

158) PORTC &= ~(1<<RELAY_PIN);
159) UART_Sendstring("Relay OFF");
    }
160) else if (received == 3){
161) UART_SendVariableValue(distance);
    }
162) prev_received = 0;
    }
163) if(received == 0){
164) menu = menu1;
165) prev_received = 0;
    }
166) break;
    } // End switch

```

خط ۱۳۵ شروع مد بلوتوث است ، در واقع دستورات case menu2 برای مد بلوتوث است.

از خط ۱۳۶ تا خط ۱۵۲ کاملاً مشابه مد التراسونیک است با این تفاوت که رله با فواصل مختلف فعال و یا غیر فعال نمی شود و کنترل رله با بلوتوث است (از خط ۱۵۳ تا ۱۶۶) در خط ۱۵۳ شرط این است که اگر متغیر prev_received برابر عدد ۱ شد دستورات کنترل رله اجرا شود (زمانی prev_received یک می شود که ما یک از کاراکتر های ۱ تا ۳ را ارسال کنیم) (هدف از استفاده ی prev_received این است که وقتی ما به عنوان مثال عدد ۱ را ارسال می کنیم فقط یک بار پیام Relay ON برای ما ارسال شود و به صورت بی نهایت ارسال نشود) (در واقع اگر از prev_received استفاده نکنیم و کاراکتر ۱ را ارسال کنیم در تابع ISR خط ۵۰ در دستور switch ، 'case 1' انتخاب می شود و در متغیر received عدد ۱ قرار داده می شود و چون تابع if ما در حلقه while قرار دارد و متغیری نداریم که عدد ۱ متغیر received را پاک کند مدام به صورت بی نهایت پیام relay ON برای من ارسال می شود. برای همین برای جلوگیری از این خطا یک متغیری به نام prev_received تعریف کردم و آن را در شرطی قرار دادم و آخر آن شرط متغیر prev_received را برابر صفر گذاشتم تا از loop در بیاید).

در خط های ۱۵۴ تا ۱۵۶ با ارسال عدد ۱ رله فعال می شود.

در خط های ۱۵۷ تا ۱۵۹ با ارسال عدد ۲ رله غیر فعال می شود.

در خط های ۱۶۰ و ۱۶۱ با ارسال عدد ۳ فاصله ای که التراسونیک تا جسم اندازه گرفته برای من ارسال

می شود (با استفاده از تابع UART_SendVariableValue که در خط ۸۵ آن را توضیح دادم).

در خط ۱۶۲ prev_received ریست می شود تا بتوانیم دوباره اعداد مورد نظر را ارسال کنیم.

در خط ۱۶۳ تا ۱۶۵ شرط این است که اگر عدد ارسال برابر 0 بود به مد التراسونیک (menu1) برود.

در خط ۱۶۶ دستور break باعث خارج شدن از مد بلوتوث می شود.

```
167) if(check >= 31){
168) dtostrf(distance, 6, 2, string);
169) LCD_String_xy(0, 0, "Distance:");
170) LCD_String_xy(0, 8, string);
171) LCD_String_xy(0, 14, "Cm");
172) check = 0;
}
}
}
```

در خط ۱۶۷ شرط این است که هر ثانیه مقادیر برنامه (نمایش فاصله) در lcd بروزرسانی شود . (در واقع

اگر بخواهیم هر ثانیه این اتفاق بیوفتد کافیت که در فرمول زیر که در قسمت های قبل هم به آن اشاره

کردم به جای متغیر Time عدد ۱ را جایگذاری کنیم :

$$\text{Check} = \text{Time} * ((F_{\text{CPU}})/(256 * 1024))$$

با جایگذاری متوجه می شویم که عدد بدست آمده تقریباً برابر ۳۱ است.

در خط ۱۶۸ با دستور dtostrf اعداد از نوع اعشاری (float or double) به رشته تبدیل می شوند تا در lcd

نمایش داده شوند . مقدار متغیر distance که مقدار فاصله در آن قرار دارد داخل رشته string قرار داده می

شود . (عدد ۶ در تابع سایز متغیر و عدد ۲ در تابع تعداد اعشار های متغیر است).

در خط ۱۶۹ با تابع LCD_String_xy می توانیم در جایگاه دلخواه lcd رشته ها را نمایش بدهیم

که در اینجا در سطر اول و ستون اول lcd عبارت "Distance:" می‌خواهیم نمایش دهیم.

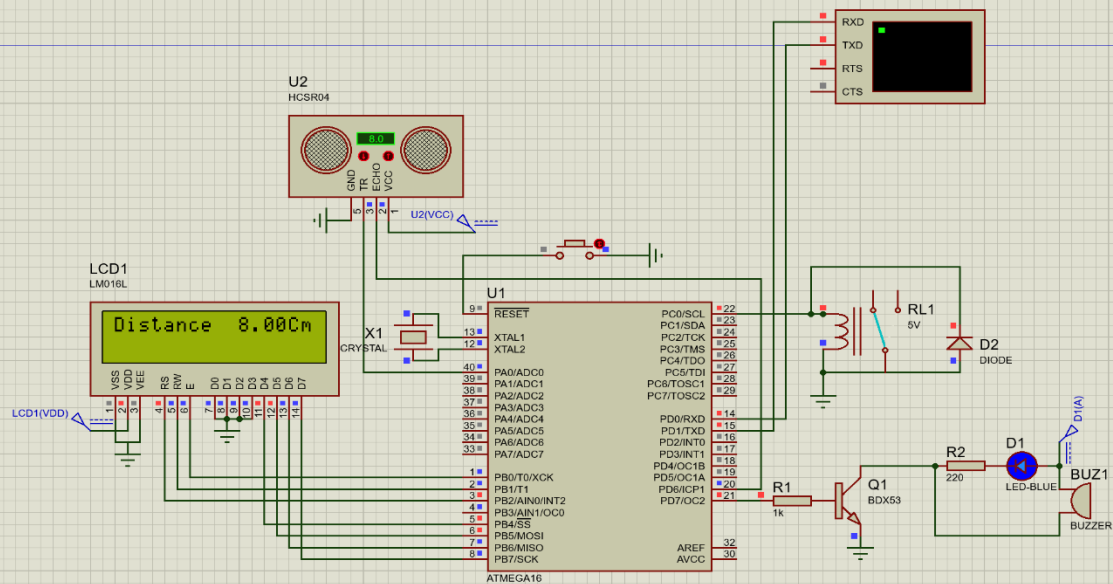
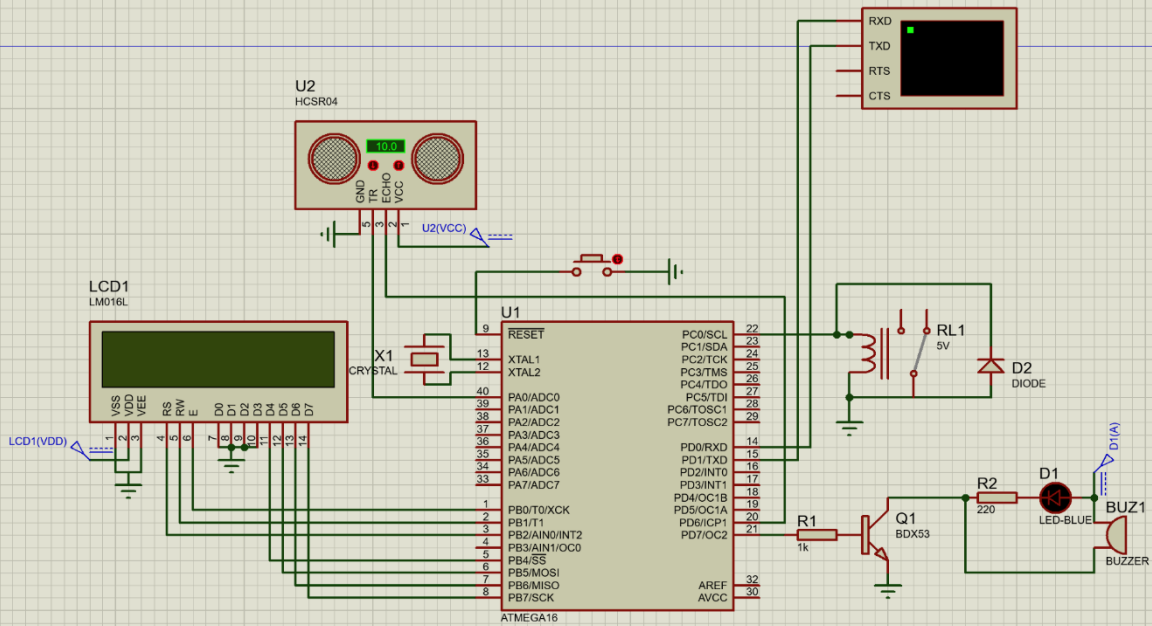
در خط ۱۷۰ می‌خواهیم در جلوی عبارت Distance: مقدار فاصله را در LCD نمایش دهیم که مقدار فاصله را در سطر اول و ستون ۸ گذاشتیم.

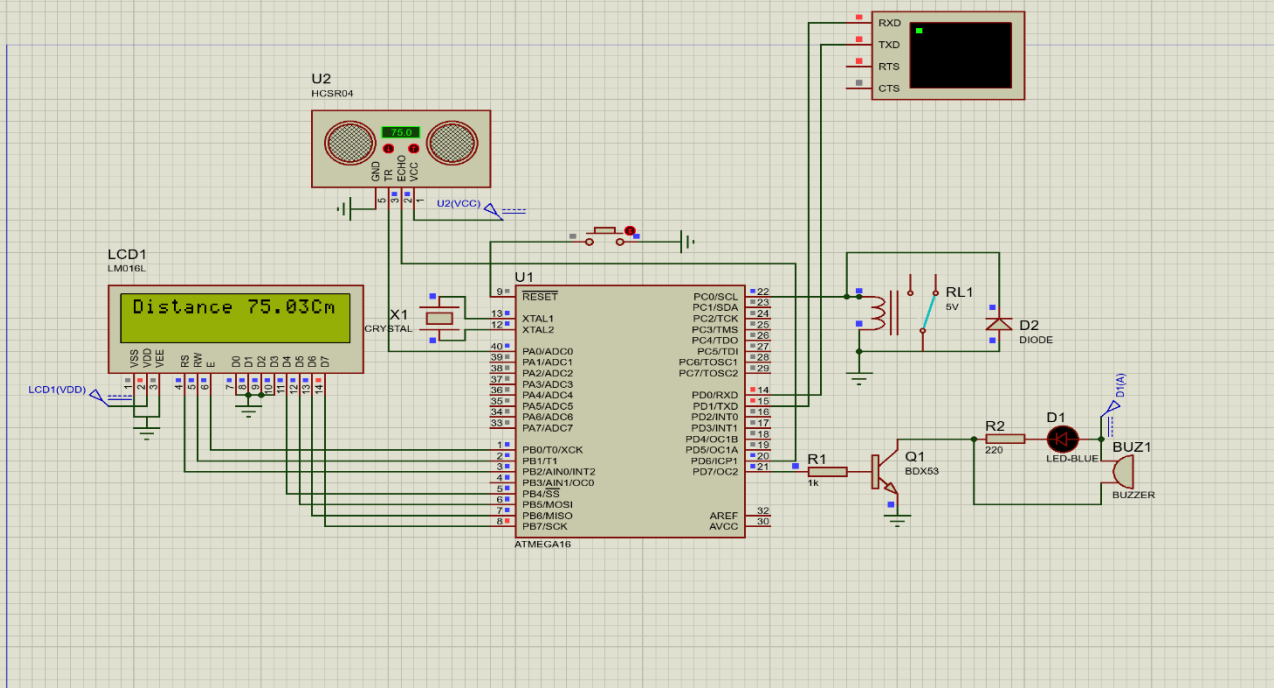
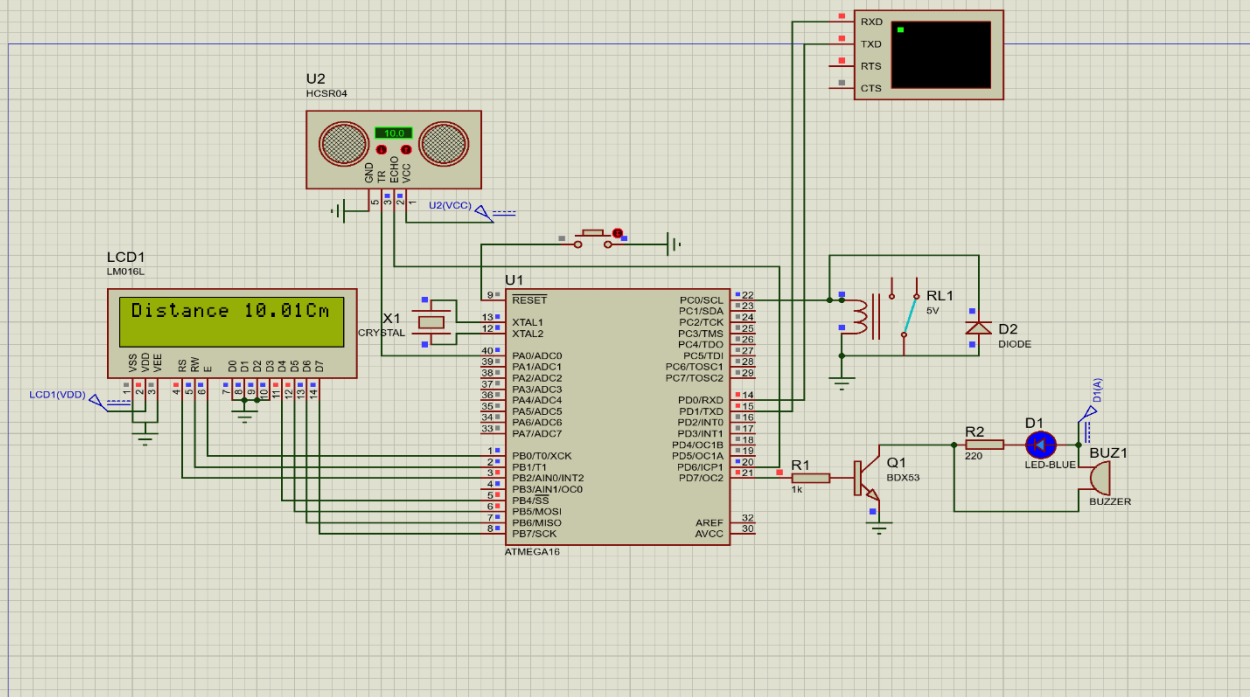
در خط ۱۷۱ می‌خواهیم عبارت Cm در جلوی مقدار فاصله نمایش دهیم. پس آن را در سطر ۱ و ستون ۱۴ می‌گذاریم.

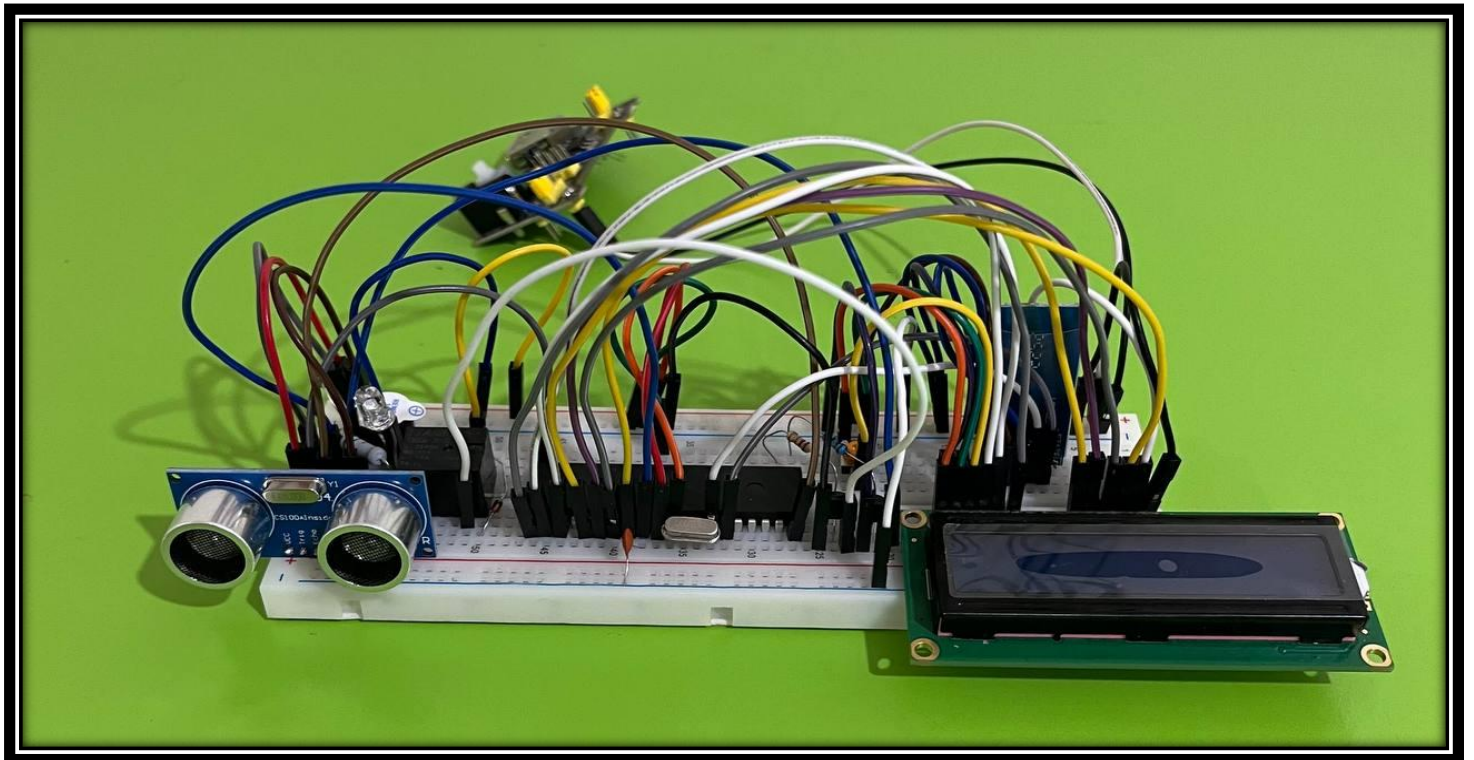
و در خط ۱۷۲ متغیر check را ریست می‌کنیم که برای بروزرسانی بعدی آماده شود.

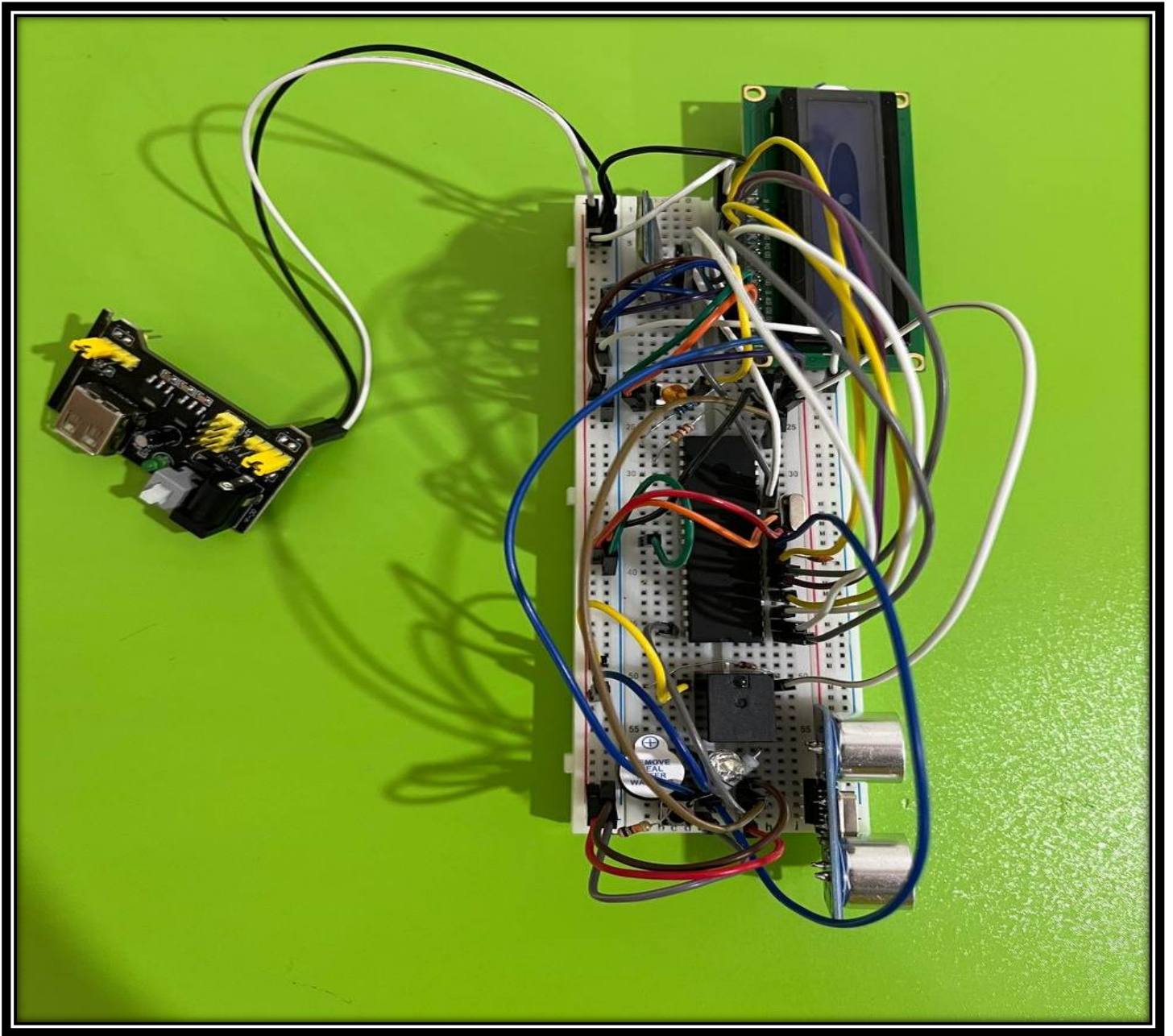
فصل دوم

سخت افزار





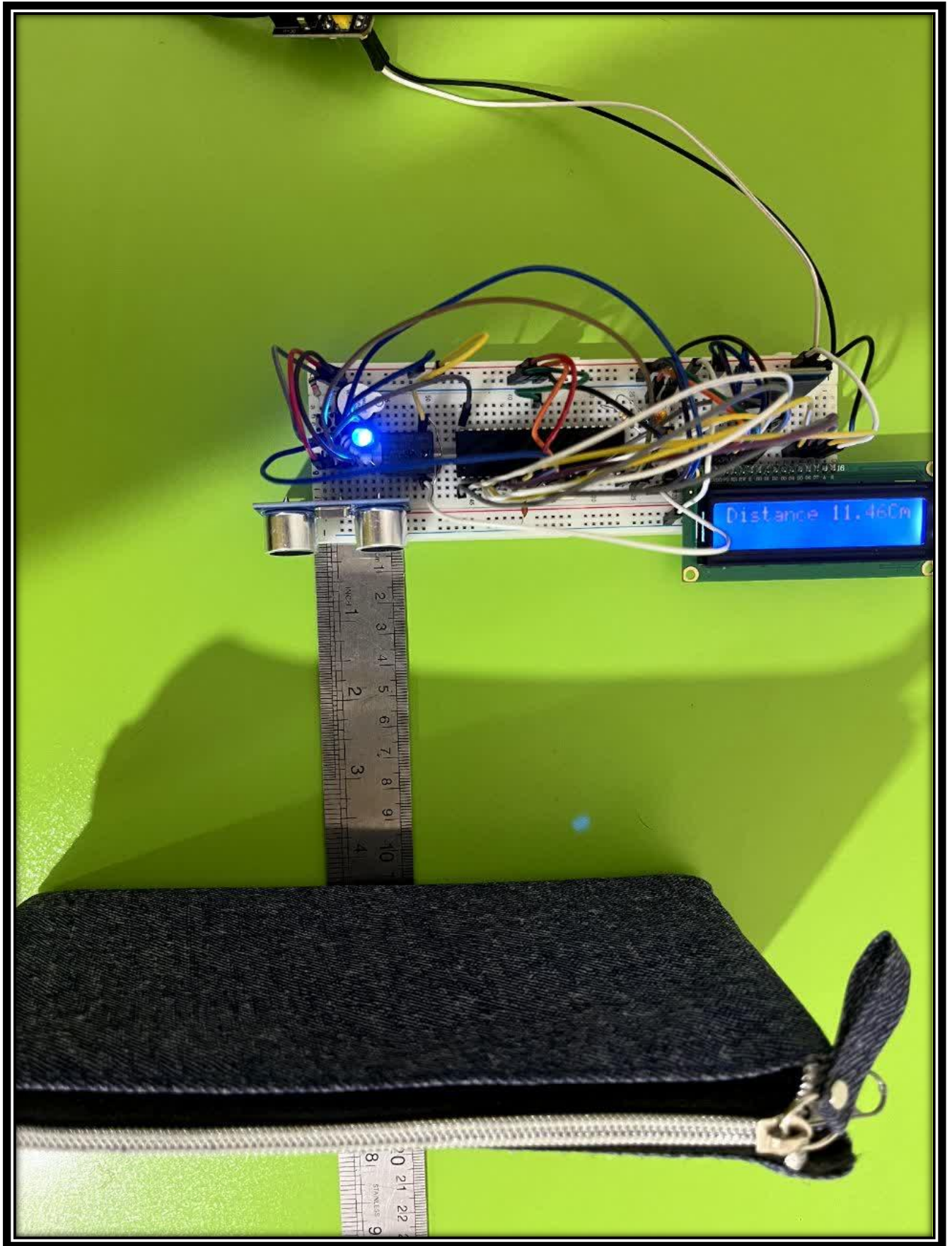


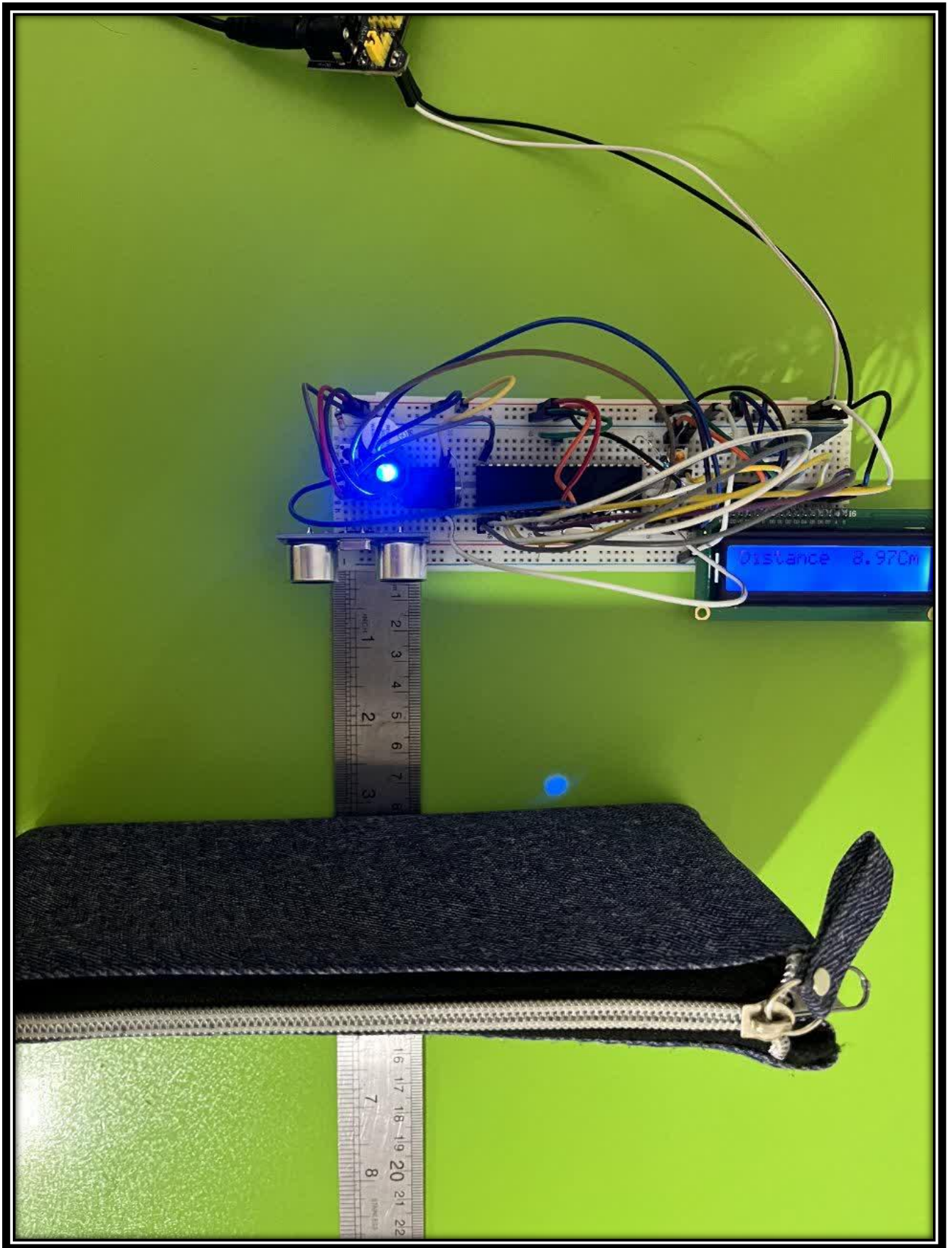


فصل سوم

نتایج







پیوست: کد نوشته شده برای پروژه به صورت یکپارچه

```
#define F_CPU 8000000UL
#include <avr/io.h>
#include <util/delay.h>
#include "LCD.h"
#include <stdlib.h> //dtostrf
#include <stdio.h> //sprintf function
#include <avr/interrupt.h>

#define BUZZER_PIN PORTD7
#define RELAY_PIN PORTC0
#define TRIG_PIN PORTA0
#define ECHO_PIN PORTD6
#define USART_BAUDRATE 9600
#define BAUD (((F_CPU / (USART_BAUDRATE*16UL)))-1)

enum {menu1,menu2}menu;
float distance;
int duration;
char string[20];
volatile int received = 0;
volatile int bluetooth_mode = 1;
volatile int overflow_counter = 0;
volatile int check = 0;
volatile char received_char = '0';
volatile char transmit_buffer[256];
volatile uint8_t transmit_read_pos = 0;
volatile uint8_t transmit_write_pos = 0;
volatile int prev_received = 0;

void init_ports() {
    DDRA |= (1 << TRIG_PIN); // Set TRIG_PIN as output on PORTA
    DDRD |= (1 << BUZZER_PIN); // Set BUZZER_PIN as output on PORTD
    DDRD &= ~(1 << ECHO_PIN); // Set ECHO_PIN as input on PORTD
    PORTD |= (1 << ECHO_PIN); // Pull-up on ECHO_PIN
    DDRC |= (1<<RELAY_PIN);
    DDRC |= (1 << PORTC1); // LED out
}

void trigger_sensor() {
    PORTA |= (1 << TRIG_PIN);
    _delay_us(10);
    PORTA &= ~(1 << TRIG_PIN);
}

unsigned int read_sensor() {
```

```

    unsigned int count = 0;

    while (!(PIND & (1 << ECHO_PIN)));

    while (PIND & (1 << ECHO_PIN)) {
        count++;
        _delay_us(0.3);
    }
    return count;
}

void init_timer0() {
    TCCR0 |= (1 << CS02) | (1 << CS00);
    TIMSK |= (1 << TOIE0);
    sei();
}

ISR(TIMER0_OVF_vect) {
    overflow_counter++;
    check++;
}

ISR(USART_RXC_vect) {
    received_char = UDR;
    switch (received_char){
        case '1':
            received = 1;
            prev_received = 1;
            break;
        case '2' :
            received = 2;
            prev_received = 1;
            break;
        case '3' :
            received = 3;
            prev_received = 1;
            break;
        case '0' :
            received = 0;
            break;
    }
}

ISR(USART_UDRE_vect) {
    if (transmit_read_pos != transmit_write_pos) {
        UDR = transmit_buffer[transmit_read_pos];
        transmit_read_pos++;

        if (transmit_read_pos >= sizeof(transmit_buffer)) {
            transmit_read_pos = 0;
        }
    } else {
        UCSRB &= ~(1 << UDRIE);
    }
}

```

```

    }
}

void UART_Txchar(char ch) {
    transmit_buffer[transmit_write_pos] = ch;
    transmit_write_pos++;

    if (transmit_write_pos >= sizeof(transmit_buffer)) {
        transmit_write_pos = 0;
    }

    UCSRB |= (1 << UDRIE);
}

void UART_Sendstring(char *str) {
    while (*str) {
        UART_Txchar(*str++);
    }
}

void UART_SendVariableValue(unsigned int variable) {
    char buffer[10];
    sprintf(buffer, "%d", variable); // Convert variable value to string
    and store in buffer
    UART_Sendstring(buffer); // Send string by UART_SendVariableValue
    Function
}

void UART_Init() {
    UCSRB |= (1 << RXEN) | (1 << TXEN);
    UCSRC |= (1 << URSEL) | (1 << UCSZ0) | (1 << UCSZ1);
    UBRRL = BAUD;
    UBRRH = (BAUD >> 8);

    UCSRB |= (1 << RXCIE);
    UCSRB |= (1 << UDRIE);
}

int main(void) {
    init_ports();
    init_timer0();
    UART_Init();
    PORTD |= (1<<BUZZER_PIN);
    LCD_Init();
    while (1) {
        trigger_sensor();
        duration = read_sensor();
        distance = (duration * (0.0343)) / 2;

        switch(menu){
        case menu1:

            if (distance > 70) {

```

```

        PORTD &= ~(1 << BUZZER_PIN);
        PORTC &= ~(1<<RELAY_PIN);
        overflow_counter = 0;
    } else if (distance <= 70 && distance > 55) {
    if (overflow_counter >= 30) { // 1 second
        PORTD ^= (1 << BUZZER_PIN);
        PORTC &= ~(1<<RELAY_PIN);
        overflow_counter = 0;
    }
    } else if (distance <= 55 && distance > 30) {
    if (overflow_counter >= 15) { // 0.5 second
        PORTD ^= (1 << BUZZER_PIN);
        PORTC &= ~(1<<RELAY_PIN);
        overflow_counter = 0;
    }
    } else if (distance <= 30 && distance > 10) {
    if (overflow_counter >= 3) { // 0.1 second
        PORTD ^= (1 << BUZZER_PIN);
        PORTC &= ~(1<<RELAY_PIN);
        overflow_counter = 0;
    }
    } else if (distance <= 10) {
    PORTD |= (1 << BUZZER_PIN);
    if(overflow_counter >= 60){
    PORTC |= (1<<RELAY_PIN);
    overflow_counter = 0;
    }
    }
    if(received != 0){
        menu=menu2;
    }
    break;

case menu2:
if (distance > 70) {
    PORTD &= ~(1 << BUZZER_PIN);
    overflow_counter = 0;
} else if (distance <= 70 && distance > 55) {
    if (overflow_counter >= 30) { // 1 second
        PORTD ^= (1 << BUZZER_PIN);
        overflow_counter = 0;
    }
    } else if (distance <= 55 && distance > 30) {
    if (overflow_counter >= 15) { // 0.5 second
        PORTD ^= (1 << BUZZER_PIN);
        overflow_counter = 0;
    }
    } else if (distance <= 30 && distance > 10) {
    if (overflow_counter >= 3) { // 0.1 second
        PORTD ^= (1 << BUZZER_PIN);
        overflow_counter = 0;
    }
    } else if (distance <= 10) {

```

```

        PORTD |= (1 << BUZZER_PIN);
    }
    if(prev_received == 1){
        if (received == 1){
            PORTC |= (1<<RELAY_PIN);
            UART_Sendstring("Relay ON");
        }
        else if (received == 2){
            PORTC &= ~(1<<RELAY_PIN);
            UART_Sendstring("Relay OFF");
        }

        else if (received == 3){
            UART_SendVariableValue(distance);
        }
        prev_received = 0;
    }
    if(received == 0){
        menu = menu1;
        prev_received = 0;
    }
    break;
} // switch

if(check >= 31){
    check = 0;
    dtostrf(distance, 6, 2, string);
    LCD_String_xy(0, 0, "Distance:");
    LCD_String_xy(0, 8, string);
    LCD_String_xy(0, 14, "Cm");
}

}

}

```